

## Basic Information

|                |                                                                         |
|----------------|-------------------------------------------------------------------------|
| Experiment No. | 02                                                                      |
| Date           | 17/09/2026                                                              |
| Title          | UPI Payment System using Interface, Encapsulation and Custom Exceptions |

## Objective / Aim

To design and implement a UPI Payment System in Java using interfaces, classes and objects, demonstrating encapsulation, abstraction through interfaces, constructors, method overriding, custom exception handling, validation, wallet balance management, and payment processing.

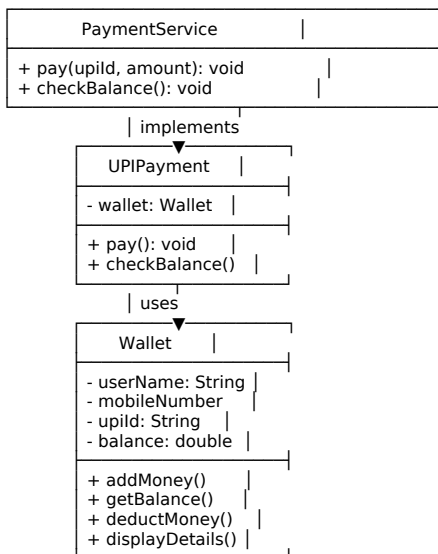
## System Requirements

|          |                                                                                                                |
|----------|----------------------------------------------------------------------------------------------------------------|
| Hardware | Standard PC/Laptop; Intel Core i3/i5 processor or equivalent; minimum 4 GB RAM; minimum 500 MB free storage    |
| Software | Windows / Linux / macOS; JDK 17 or JDK 21; Eclipse, IntelliJ IDEA, NetBeans, VS Code, or another Java IDE; Com |

## Algorithm / Logic Description

1. Create a PaymentService interface containing pay() and checkBalance() methods.
2. Create custom exception classes: InsufficientBalanceException, InvalidUPIException, and InvalidAmountException.
3. Create a Wallet class with private userName, mobileNumber, upild, and balance attributes to demonstrate encapsulation.
4. Initialize the wallet balance to zero using the Wallet constructor.
5. Define addMoney() to add a positive amount to the wallet.
6. Define getBalance() to retrieve the current balance.
7. Define deductMoney() to reduce the wallet balance after a successful payment.
8. Define displayWalletDetails() to display user, mobile, UPI ID, and balance.
9. Create UPIPayment implementing PaymentService and associate it with a Wallet object.
10. In pay(), validate the receiver UPI ID; if invalid, throw InvalidUPIException.
11. Validate the payment amount; if zero or negative, throw InvalidAmountException.
12. Check sufficient balance; if insufficient, throw InsufficientBalanceException.
13. If validations pass, deduct the payment amount and display payment success.
14. Use catch blocks for each custom exception and finally to show transaction completion.
15. In main(), create the wallet, add money, check balance, perform valid and invalid payments, and display final details.

## UML Class Diagram



Custom Exceptions:  
• InsufficientBalanceException  
• InvalidUPIException  
• InvalidAmountException

Relationship: UPIPayment implements the PaymentService interface and uses a Wallet object. The custom exception classes extend Exception and are used for transaction validation.

## Source Code

```
package javapractice;
```

```

// PaymentService Interface
interface PaymentService {

    void pay(String upild, double amount);

    void checkBalance();
}

// Custom Exception for insufficient balance
class InsufficientBalanceException extends Exception {

    public InsufficientBalanceException(String message) {
        super(message);
    }
}

// Custom Exception for invalid UPI ID
class InvalidUPIException extends Exception {

    public InvalidUPIException(String message) {
        super(message);
    }
}

// Custom Exception for invalid amount
class InvalidAmountException extends Exception {

    public InvalidAmountException(String message) {
        super(message);
    }
}

// Wallet Class
class Wallet {

    // Private data members to demonstrate encapsulation
    private String userName;
    private String mobileNumber;
    private String upild;
    private double balance;

    // Constructor
    public Wallet(String userName, String mobileNumber, String upild) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upild = upild;
        this.balance = 0.0;
    }

    // Method to add money
    public void addMoney(double amount) {

        if (amount > 0) {
            balance = balance + amount;
            System.out.println("Money added successfully: ₹" + amount);
        } else {
            System.out.println("Invalid amount.");
        }
    }

    // Method to get balance
    public double getBalance() {
        return balance;
    }

    // Method to deduct money
    public void deductMoney(double amount) {
        balance = balance - amount;
    }

    // Method to display wallet details
    public void displayWalletDetails() {

        System.out.println("\n----- Wallet Details -----");
        System.out.println("User Name   : " + userName);
        System.out.println("Mobile Number: " + mobileNumber);
        System.out.println("UPI ID      : " + upild);
        System.out.println("Balance     : ₹" + balance);
    }
}

// UPI Payment Class
class UPIPayment implements PaymentService {

    private Wallet wallet;

    // Constructor
    public UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }

    // Pay method
    @Override
    public void pay(String upild, double amount) {

        try {

            // Validate UPI ID
            if (upild == null ||

```

```

        !upild.contains("@") ||
        upild.startsWith("@") ||
        upild.endsWith("@")) {

            throw new InvalidUPIException(
                "Invalid UPI ID: " + upild
            );
        }

        // Validate payment amount
        if (amount <= 0) {

            throw new InvalidAmountException(
                "Payment amount must be greater than zero."
            );
        }

        // Check sufficient balance
        if (amount > wallet.getBalance()) {

            throw new InsufficientBalanceException(
                "Insufficient balance for this payment."
            );
        }

        // Deduct payment amount
        wallet.deductMoney(amount);

        System.out.println("\nPayment Successful!");
        System.out.println("Receiver UPI ID: " + upild);
        System.out.println("Amount Paid    : ₹" + amount);
    }

    catch (InvalidUPIException e) {

        System.out.println("Transaction Failed: "
            + e.getMessage());
    }

    catch (InvalidAmountException e) {

        System.out.println("Transaction Failed: "
            + e.getMessage());
    }

    catch (InsufficientBalanceException e) {

        System.out.println("Transaction Failed: "
            + e.getMessage());
    }

    finally {

        System.out.println("Transaction processing completed.");
    }
}

// Check balance method
@Override
public void checkBalance() {

    System.out.println("Available Wallet Balance: ₹" + wallet.getBalance());
}

// Main Class
public class UPIPaymentSystem {

    public static void main(String[] args) {

        // Create Wallet object
        Wallet wallet = new Wallet(
            "riya",
            "9456543210",
            "riya@upi"
        );

        // Display initial wallet details
        wallet.displayWalletDetails();

        // Add money to wallet
        wallet.addMoney(5000);

        // Create UPI Payment object
        PaymentService payment = new UPIPayment(wallet);

        // Check balance
        payment.checkBalance();

        // Make a valid payment
        System.out.println("\n--- Valid Payment ---");
        payment.pay("rahul@upi", 1500);

        // Check balance after payment
        payment.checkBalance();

        // Invalid UPI test
        System.out.println("\n--- Invalid UPI Test ---");
    }
}

```

```

payment.pay("rahulupi", 500);

// Invalid amount test
System.out.println("\n--- Invalid Amount Test ---");
payment.pay("rahul@upi", -100);

// Insufficient balance test
System.out.println("\n--- Insufficient Balance Test ---");
payment.pay("rahul@upi", 5000);

// Display final wallet details
wallet.displayWalletDetails();
}
}

```

## Compilation & Execution Commands

Save the source code exactly as UPIPaymentSystem.java inside the javapractice folder because the program uses the package declaration package javapractice;

```

javac javapractice/UPIPaymentSystem.java
java javapractice.UPIPaymentSystem

```

## Output Screen

```

----- Wallet Details -----
User Name   : riya
Mobile Number: 9456543210
UPI ID      : riya@upi
Balance     : ₹0.0
Money added successfully: ₹5000.0
Available Wallet Balance: ₹5000.0

--- Valid Payment ---

Payment Successful!
Receiver UPI ID: rahul@upi
Amount Paid    : ₹1500.0
Transaction processing completed.
Available Wallet Balance: ₹3500.0

--- Invalid UPI Test ---
Transaction Failed: Invalid UPI ID: rahulupi
Transaction processing completed.

--- Invalid Amount Test ---
Transaction Failed: Payment amount must be greater than zero.
Transaction processing completed.

--- Insufficient Balance Test ---
Transaction Failed: Insufficient balance for this payment.
Transaction processing completed.

----- Wallet Details -----
User Name   : riya
Mobile Number: 9456543210
UPI ID      : riya@upi
Balance     : ₹3500.0

```

## Result

The UPI Payment System was successfully implemented in Java using an interface, classes and objects. The program demonstrates encapsulation, abstraction through the PaymentService interface, constructors, custom exception handling, input validation, wallet balance management, successful payment processing, and handling of invalid UPI IDs, invalid amounts, and insufficient balance.